Concrete Site

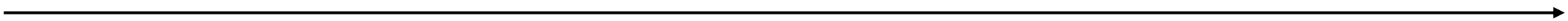
by Philippe Beauchemin

Concept

The viewer is prompted to use their personal device to access the project; in doing so, they unconsciously provide information to be extracted.

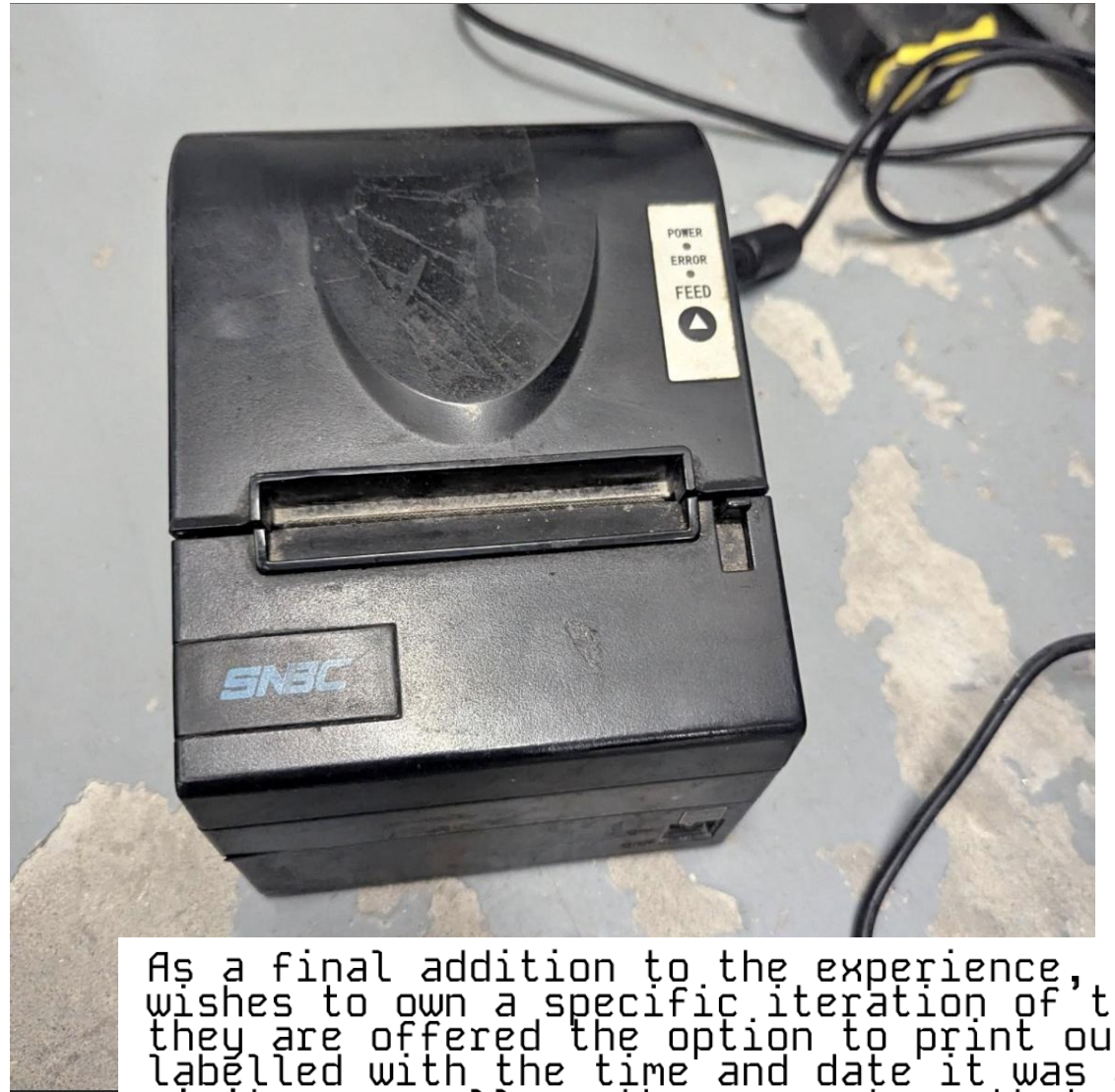
The project displays an image of a concrete wall, procedurally generated, which is altered by the viewer's presence on the virtual site. The concrete wall reacts to various components of the viewer's data, changing its visual qualities accordingly.

As viewers join the site, the concrete wall changes over time and informs its viewers of these changes through a timeline.



As each aspect of the visitor's data is mapped onto a specific visual quality of the image, ***Concrete Site*** **translates how this data correlates to the viewer's material conditions in the outside world.** ***Concrete Site*** **critiques the materiality of concrete in relation to its socio-cultural and historical context, while also offering a critique of class and e-tourism in the age of surveillance capitalism.**

As a final addition to the experience, if the viewer wishes to own a specific iteration of the concrete wall, they are offered the option to print out the image, labelled with the time and date it was affected by a visitor as well as the parameters that led to the iteration (mimicking a Pantone swatch).



As a final addition to the experience, if the viewer wishes to own a specific iteration of the concrete wall, they are offered the option to print out the image, labelled with the time and date it was affected by a visitor as well as the parameters that led to the iteration (mimicking a Pantone swatch).



SNBC Thermal POS Receipt Printer (BTP-R880NP)

[redacted]$

Message envoyé au/à la vendeur(se)

Voir la conversation

Description

ink needed). Voir plus

Algorithmic Approach

I plan to use the techniques of **procedural texture generation, procedural rhetoric and emergent narrative.**

{Procedural texture generation} The visual component of the piece, running in real-time over a long period of time and reacting to users' data as well as API data

{Procedural rhetoric} Pessimistic notions of concrete as a material (Criticizing its relationship to power, wealth and control)

{Emergent narrative} The user explores the various iterations of the wall over time to understand the different visual outcomes

The project will heavily rely on data extraction and mappings between:

- a user's device (model, browser, location),
- APIs (such as weather at that location), and
- the project, acting as both a client and a server for data exchanged over a network (which I will learn in CART 351: Networks and Navigation)

Algorithmic Approach

{Procedural texture generation} The visual component of the piece, running in real-time over a long period of time and reacting to users' data as well as API data

Design Research Society
DRS Digital Library

DRS Biennial Conference Series DRS2004 - Futureground

Nov 17th, 12:00 AM

Texture Synthesis Algorithm for Graphics Designers.

Meng-Dar Shieh
National Cheng Kung University

Yu-Le Lin
National Cheng Kung University

2015/06/16 v0.5.1 IPOL article class

IPOL IMAGE PROCESSING ON LINE

Published in Image Processing On Line on 2014–11–17.
Submitted on 2013–03–13, accepted on 2014–04–29.
ISSN 2105–1232 © 2014 IPOL & the authors CC–BY–NC–SA
This article is available online with supplementary materials,
software, datasets and online demo at
https://doi.org/10.5201/ipol.2014.79

The Heeger-Bergen Pyramid-Based Texture Synthesis Algorithm

Thibaud Briand[1], Jonathan Vacher[2], Bruno Galerne[3], Julien Rabin[4]

[1] Département de mathématiques, ENS Cachan, France (tbriand@ens-cachan.fr)
[2] CEREMADE, CNRS-Université Paris-Dauphine, France (jonathan.vacher@ceremade.dauphine.fr)
[3] Laboratoire MAP5 (UMR CNRS 8145), Université Paris Descartes, France (bruno.galerne@parisdescartes.fr)
[4] GREYC, CNRS-ENSICAEN, Université de Caen, France (julien.rabin@unicaen.fr)

Communicated by Yann Gousseau *Demo edited by* Enric Meinhardt-LLopis

28 Textures
29 Probabilistic Graphical Models
Generative Image Models and Representation Learning
30 Representation Learning
31 Perceptual Grouping
32 Generative Models
33 Generative Modeling Meets Representation Learning
34 Conditional Generative Models
Challenges in Learning-Based Vision
35 Data Bias and Shift
36 Training for Robustness and Generality
37 Transfer Learning and Adaptation
Understanding Geometry
38 Representing Images and Geometry
39 Camera Modeling and Calibration
40 Stereo Vision
41 Homographies
42 Single View Metrology
43 Learning to Estimate Depth from a Single Image
44 Multiview Geometry and Structure from Motion
45 Radiance Fields
Understanding Motion
46 Motion Estimation
47 3D Motion and Its 2D

28 Textures

28.1 Introduction

The visual world is made of objects (chairs, cars, tables), surfaces (walls, floor, ceiling), and stuff (wood, grass, water). This chapter is about seeing stuff [1]. How do we build image representations that can capture what makes wood different from a wall of bricks?

Representing textures is a task, similar to color perception, that is intimately related to human perception. A texture is a type of image that is composed by a set of similar looking elements. A texture representation contains information about the statistics of its constituent elements, but not about the elements individually.

In this chapter we will introduce the problems of texture analysis and synthesis as a way to explore texture representations. The models presented here are precursors of more modern approaches using deep learning. But many of the concepts, and some of the intuitions about why these models might be successful, are better understood by exploring first simple yet powerful models.

Texture synthesis can be solved by a trivial algorithm as shown in Figure 28.1. But such an algorithm, although successful in practice, will give us little understanding on how human perception works and how to modify textures to create new ones. It will also not help us in understanding what representation can be useful to measure similarity between textures.

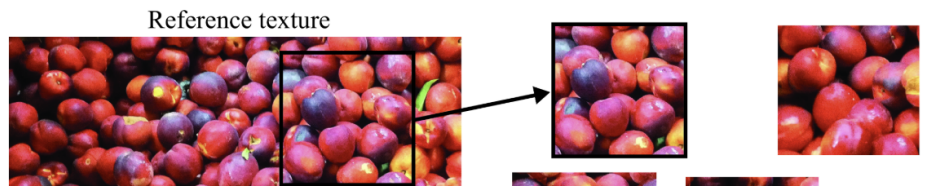
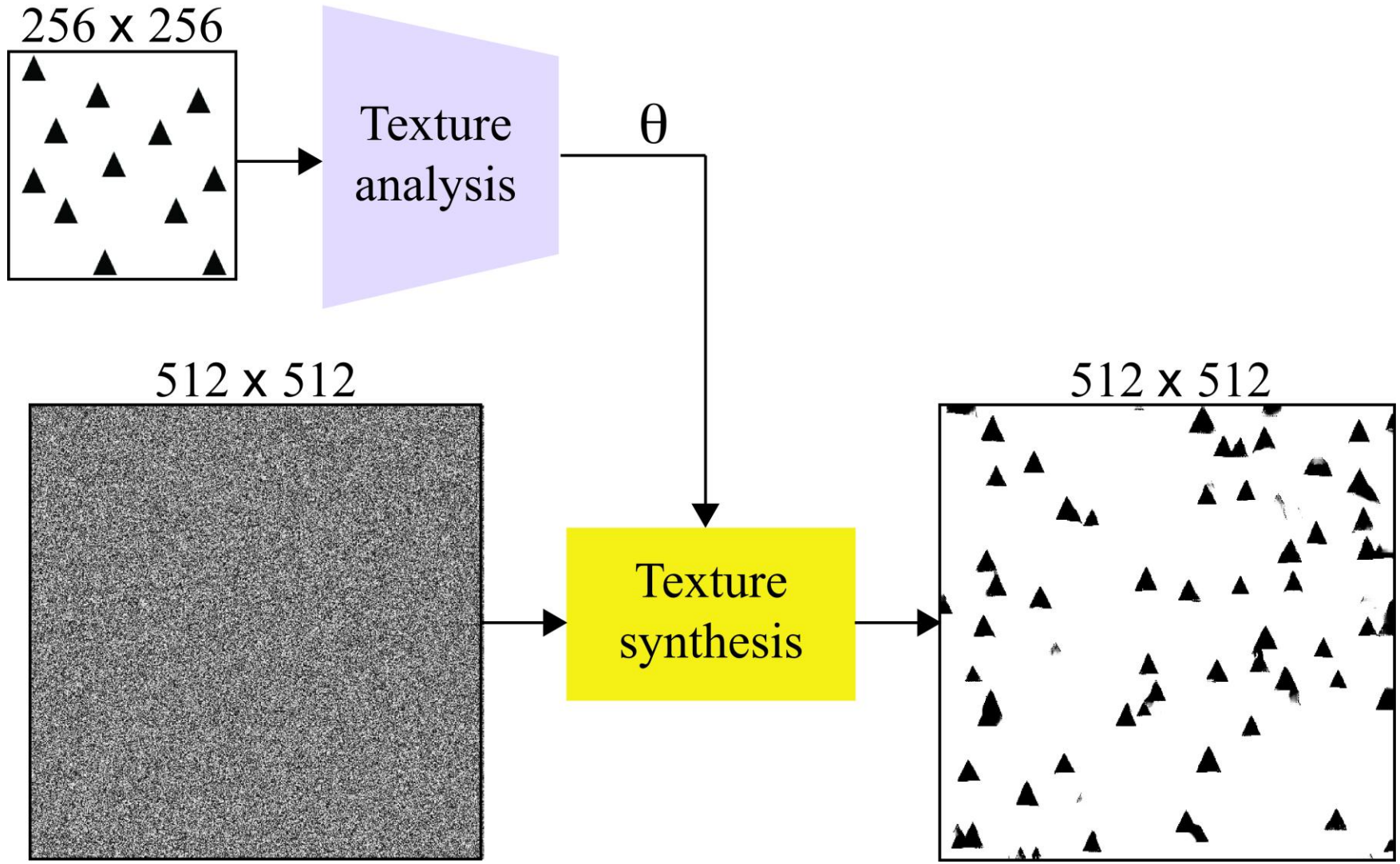


Figure 28.1: The infinite texture generation algorithm. If we had access to a very large image of the texture we want to generate, we could just crop pieces from it to create new images.

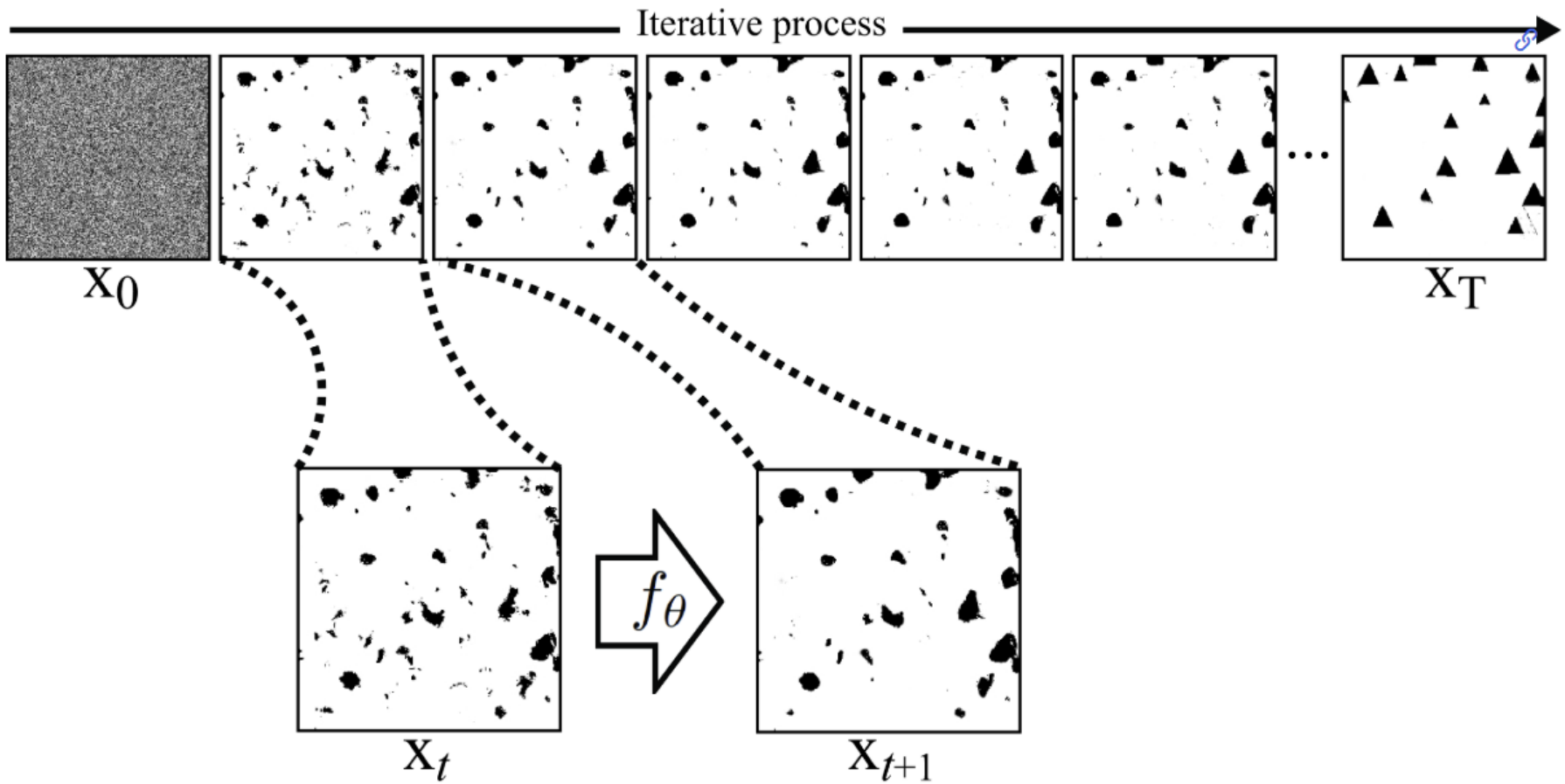
Table of contents
28.1 Introduction
28.2 A Few Notes about Human Perception
28.3 Heeger-Bergen Texture Analysis and Synthesis
28.4 Efros-Leung Texture Analysis and Synthesis Model
28.5 Connection to Deep Generative Models
28.6 Concluding Remarks

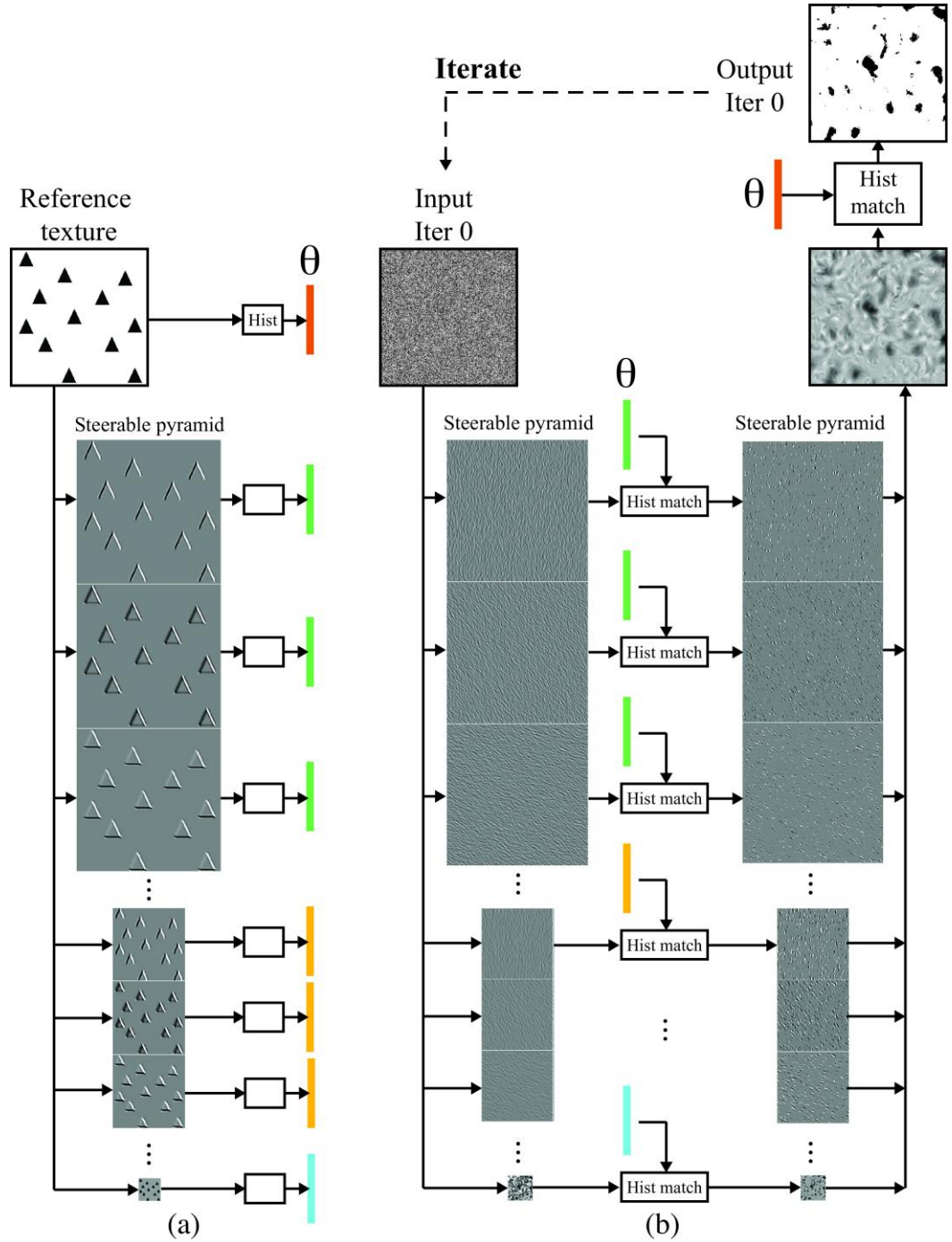
Edit this page
Report an issue
Accessibility

28.3 Heeger-Bergen Texture Analysis and Synthesis



Texture synthesis is usually an iterative image synthesis procedure that begins with a random noise image and uses the parameters θ to produce a new improved image as shown in Figure 28.6. We will describe a similar algorithm in Section 32.8 when talking about **diffusion models**.




Iterate
Output
Iter 0
Reference
texture
θ
Hist
Input
Iter 0
θ
Hist
match
Steerable pyramid
Steerable pyramid
Steerable pyramid
Hist match
Hist match
Hist match
Hist match
Hist match
(a)
(b)

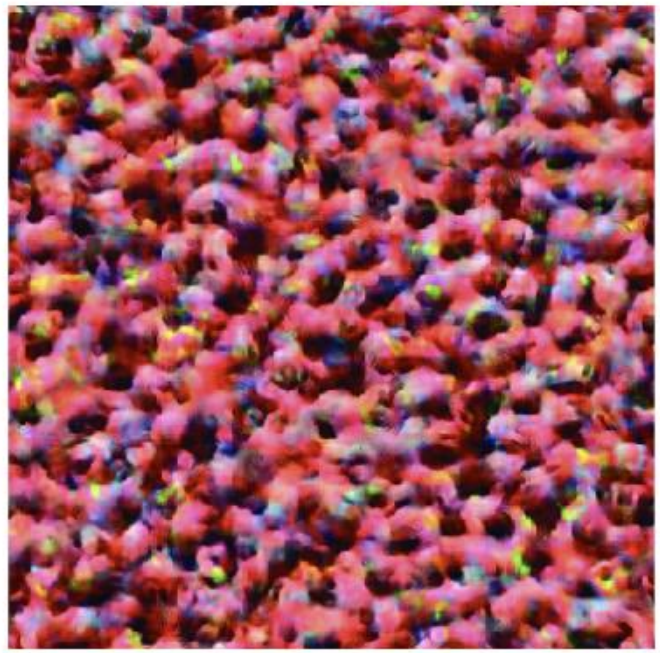
Texture models using statistics of filter outputs are a precursor to diffusion models (Section 32.8) and share a number of architectural features.

The Heeger and Bergen model was one of the first successful approaches that used filter outputs to represent arbitrary textures. Other models followed after that introducing stronger statistical representations capable of generating higher quality textures [11].

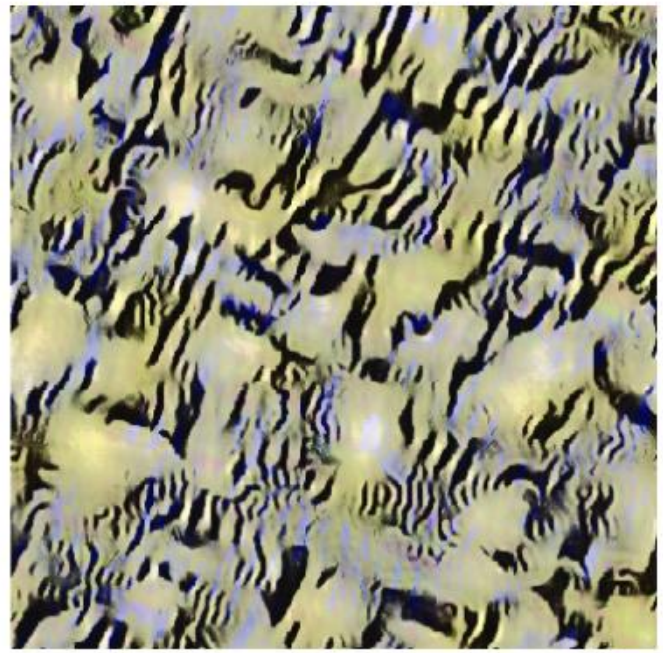
These models, usually called **parametric texture models**, stayed popular until 2000. After that, **nonparametric texture models** became dominant (Section 28.4) until generative models with neural networks emerged around 2014.

For the results in Figure 28.9, the algorithm is run only for 15 iterations. Adding more iterations makes the images look worse.









```
# Implementation of the Heeger and Bergen texture synthesis algorithm.
# 2024
#
# https://visionbook.mit.edu/textures.html

import numpy as np
import matplotlib.pyplot as plt
from skimage import io, transform
from skimage.color import rgb2gray
from skimage.exposure import match_histograms
```

```
# pyrtools is a library used for image processing, particularly useful for operations involving pyramids and wavelets.
!pip install pyrtools
import pyrtools as pt
```

```
def load_image(image_path):
    img = io.imread(image_path)
    return rgb2gray(img)  # Convert to grayscale for simplicity

def build_pyramid(image, max_levels): # this is not used, but nice to have it here for future reference
    return pt.pyramids.LaplacianPyramid(image, height=max_levels)

def build_steerable_pyramid(image, max_levels, orientations):
    return pt.pyramids.SteerablePyramidFreq(image, height=max_levels, order=orientations)

def histogram_matching(original, reference):
    matched = match_histograms(original, reference)
    return matched

def synthesis(input_texture, iterations=5, max_levels=3, orientations=6):
    original_pyramid = build_steerable_pyramid(input_texture, max_levels, orientations)
    synthesized_texture = np.random.rand(*input_texture.shape)

    for _ in range(iterations):
        # first match the histograms of the pixels distributions
        synthesized_texture = histogram_matching(synthesized_texture, input_texture)
        synth_pyramid = build_steerable_pyramid(synthesized_texture, max_levels, orientations)

        # Iterate through the scales and orientations of the pyramid. It is stored as a dictionary.
        for key, original_coeffs in original_pyramid.pyr_coeffs.items():
            synth_coeffs = synth_pyramid.pyr_coeffs[key]
            # Ensure both are numpy arrays for histogram matching
            matched = histogram_matching(synth_coeffs, original_coeffs)
            synth_pyramid.pyr_coeffs[key] = matched

        synthesized_texture = synth_pyramid.recon_pyr()

    synthesized_texture = histogram_matching(synthesized_texture, input_texture)
    return synthesized_texture
```

[ ]

```
# Load an example texture
#sample_image = load_image('http://people.csail.mit.edu/torralba/share/course/triangle.jpg')  # Replace with a valid URL or file path
sample_image = load_image('http://people.csail.mit.edu/torralba/share/course/text_rotated.jpg')  # Replace with a valid URL or file path

sample_image = transform.resize(sample_image, (256, 256))  # Resize for simplicity
plt.imshow(sample_image, cmap='gray')
plt.show()
```

[ ]

```
# Let's check the steerable pyramid decomposition of the sample image

# compute the pyramid with 2 scales and 4 orientations per scale:
original_pyramid = build_steerable_pyramid(sample_image, 2, 4)
# the next function plots the output of all the filters
pt.pyrshow(original_pyramid.pyr_coeffs);
```

[ ]

```
# Perform texture synthesis (this is a bit slow. It takes >15 seconds.)
Niterations = 100
synthesized = synthesis(sample_image, Niterations)
```

[ ]

```
# Display results
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title('Original Texture')
plt.imshow(sample_image, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title('Synthesized Texture')
plt.imshow(synthesized, cmap='gray')
plt.axis('off')

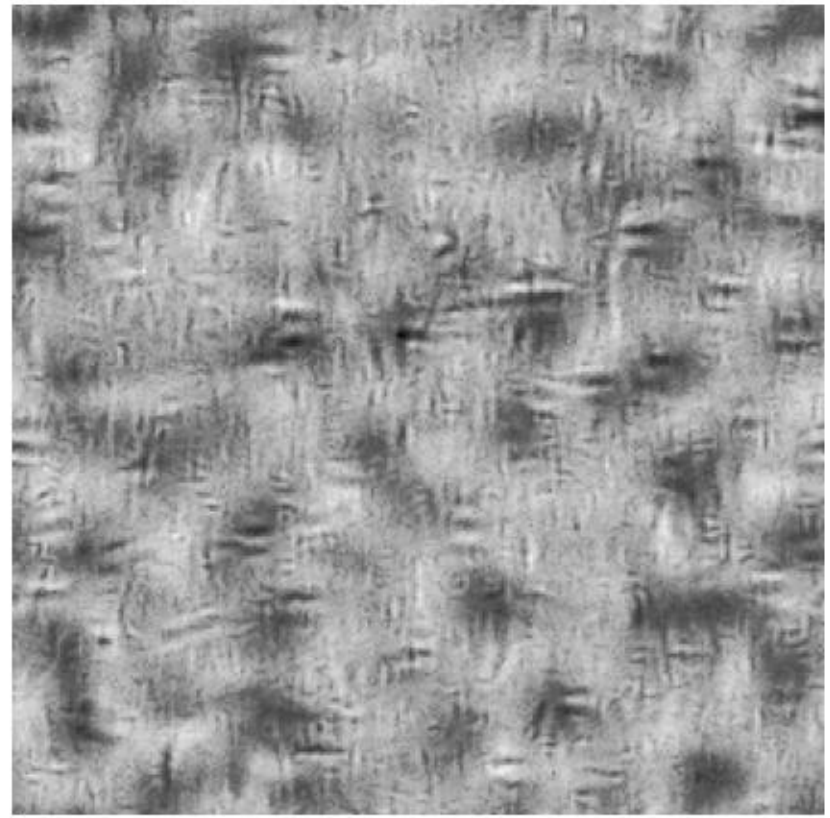
plt.show()
```



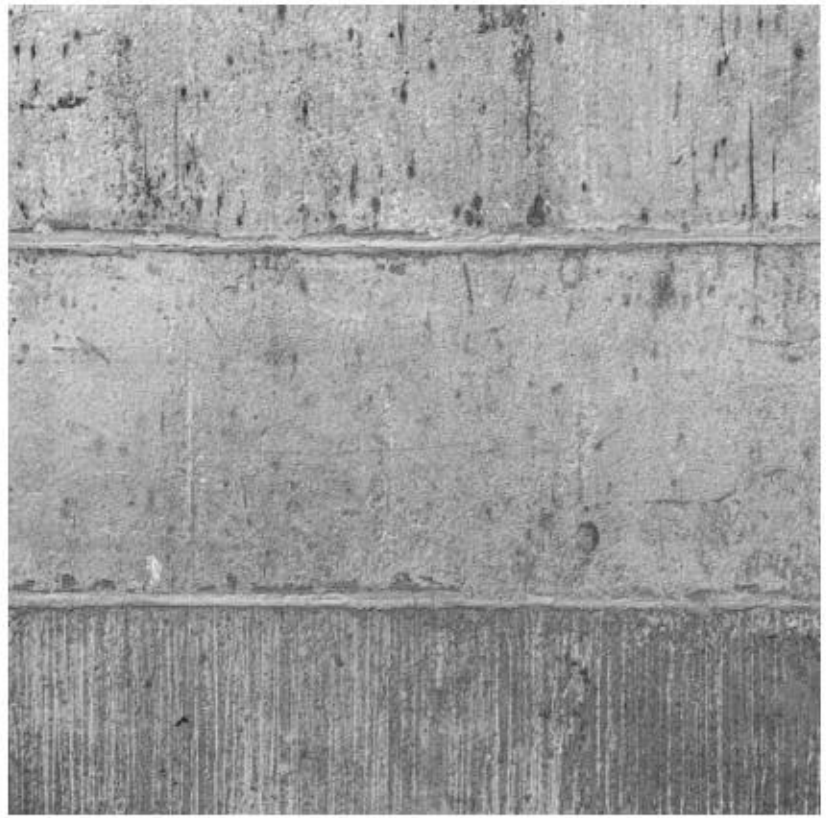
Original Texture



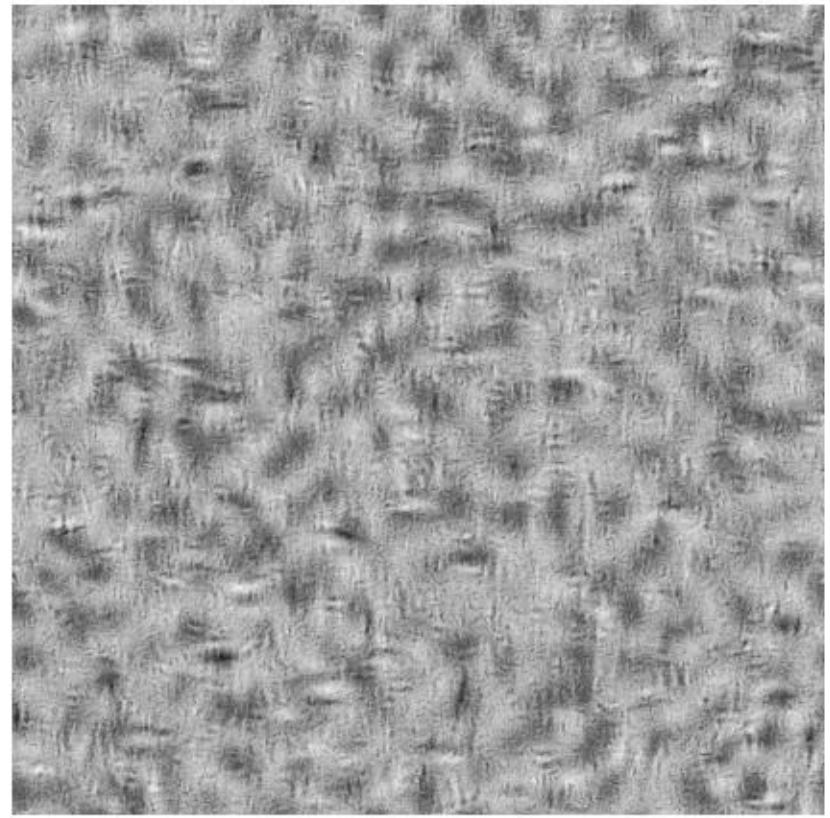
Synthesized Texture



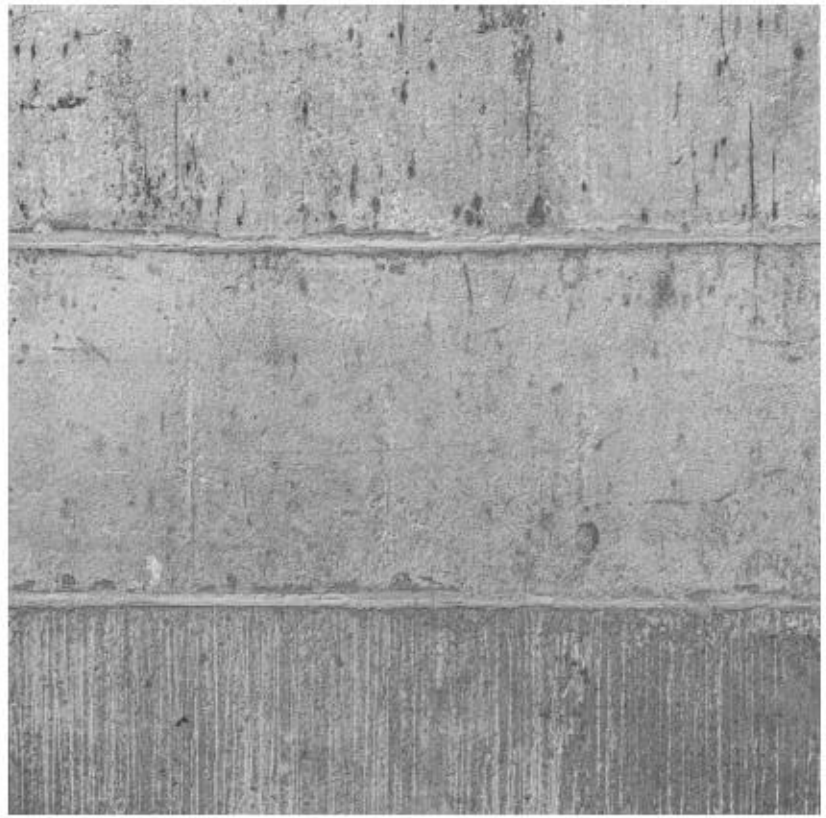
Original Texture



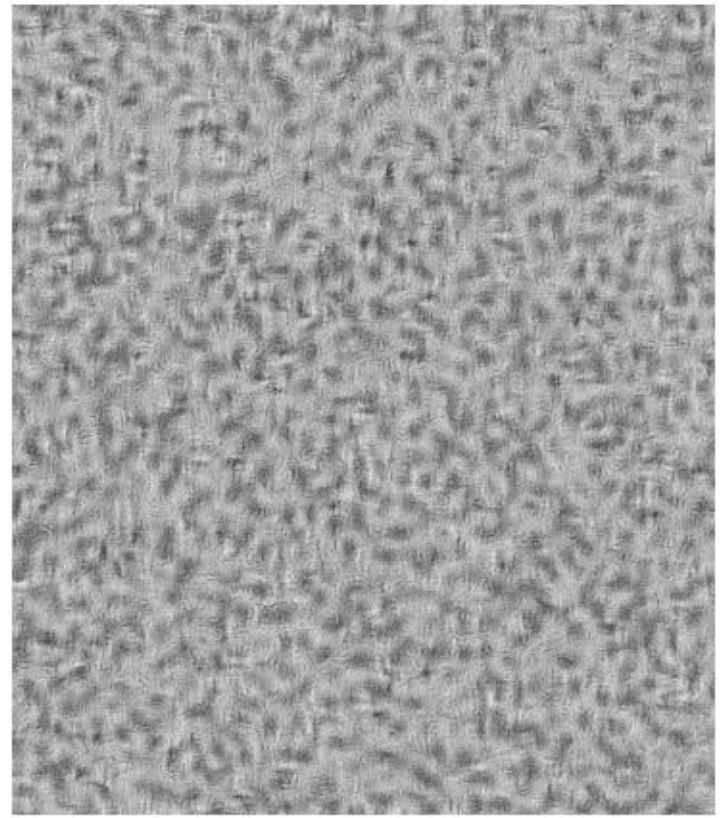
Synthesized Texture



Original Texture



Synthesized Texture



be?

How can I cause it to “decay”?

What does changing the number of iterations affect?

How much does the starting image affect the final result?

Can I add other images to the mix?

How can I introduce user data?

Should I create user “archetypes”?

Can I source the images from an API?

Tests:

- Time
- Rgba example image

Play with:

- N of iterations
- Size of desired image
- Feedback

How long should it take?

How “good” should it be?

How can I cause it to “decay”?

What does changing the number of iterations affect?

be?

How can I cause it to “decay”?

What does changing the number of iterations affect?

How much does the starting image affect the final result?

Can I add other images to the mix?

How can I introduce user data?

Should I create user “archetypes”?

Can I source the images from an API?

Scope and Feasibility

- procedural texture generation algorithms
  - I have never written one but I have written iterative algorithms before

  Not my own, but I understand it

  - I am not sure how computationally expensive it will be to keep the algorithm running over a long period of time.

  Can run it as request
- data extraction
  - I will need to better understand how to extract data and manipulate it (from users and APIs)

  Seeing this in CART 351